

Database Design for Healthcare system

Table of Contents

1. Introduction.....	3
2. Database Design.....	3
3. Data Insertion.....	5
4. Task 2 Queries: SQL Queries and Outputs.....	8
5. Task 3 Queries: Business Analytics Queries and Outputs.....	11
6. Business Insights and Applications.....	14
7. Challenges and Limitations.....	15
8. Conclusion.....	15
Reference list.....	16

1. Introduction

Healthcare data management is of great importance in delivering quality patient care, effectiveness of the health care systems and aids the administrators in their decision making process. This report is centered on creating a healthcare database in MySQL for the purpose of preparing monthly report on new patients registered in the system in the last one year. Besides, the report also covers the application of SQL queries for business analytics in order to derive insights from the patient data (Muji et al., 2009).

The purpose is to create and use a structured database that will contain patient information and enable specific analyses to produce reports on patient enrolment, patterns and future requirements for planning. The queries will give an indication of the number of patients registered per month, the pattern of doctor –patient interactions, and prescription records (Celikel Cankaya, 2019).

2. Database Design

Relational database is used in the organization where the major tables hold the necessary information to support healthcare activities and functions. The core table for this report is the Patients table that contains registration information of the patients. The structure of the table is somewhat optimized using primary keys and other attributes to reduce the problem of data integrity and query execution (C., C. and E., 2018).

Patients Table Structure:

```
4 • ○ CREATE TABLE Patients (  
5     PatientID INT AUTO_INCREMENT PRIMARY KEY,  
6     FirstName VARCHAR(50),  
7     LastName VARCHAR(50),  
8     DateOfBirth DATE,  
9     ContactInfo VARCHAR(100)  
10  );
```

Doctors Table Structure:

```
30 • ○ CREATE TABLE Doctors (  
31     DoctorID INT AUTO_INCREMENT PRIMARY KEY,  
32     FirstName VARCHAR(50),  
33     LastName VARCHAR(50),  
34     Specialty VARCHAR(100),  
35     ContactInfo VARCHAR(100)  
36  );
```

Appointments Table Structure:

```
58 • ○ CREATE TABLE Appointments (  
59     AppointmentID INT AUTO_INCREMENT PRIMARY KEY,  
60     PatientID INT,  
61     DoctorID INT,  
62     AppointmentDate DATE,  
63     FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
64     FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)  
65 );
```

MedicalRecords Table Structure:

```
86 • ○ CREATE TABLE MedicalRecords (  
87     RecordID INT AUTO_INCREMENT PRIMARY KEY,  
88     PatientID INT,  
89     RecordDate DATE,  
90     Diagnosis VARCHAR(255),  
91     Treatment VARCHAR(255),  
92     FOREIGN KEY (PatientID) REFERENCES Patients(PatientID)  
93 );
```

Medications Table Structure:

```
115 • ○ CREATE TABLE Medications (  
116     MedicationID INT AUTO_INCREMENT PRIMARY KEY,  
117     MedicationName VARCHAR(100),  
118     Dosage VARCHAR(50)  
119 );
```

Prescriptions Table Structure:

```
141 • ○ CREATE TABLE Prescriptions (  
142     PrescriptionID INT AUTO_INCREMENT PRIMARY KEY,  
143     PatientID INT,  
144     MedicationID INT,  
145     PrescriptionDate DATE,  
146     FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),  
147     FOREIGN KEY (MedicationID) REFERENCES Medications(MedicationID)  
148 );
```

3. Data Insertion

To enhance the realism of the tables, 15 records were also input to the Patients table. These records imitate new patient registrations for the period of 15 months, starting from the beginning of the year 2023 up to the end of the first quarter of the year 2024. This information will be used to produce monthly patient registration report (Donaldson and Lohr, 2019).

Sample Data for the Patients Table

```
12 • INSERT INTO Patients (FirstName, LastName, DateOfBirth, ContactInfo)
13 VALUES
14 ('John', 'Doe', '1980-05-14', 'john.doe@example.com'),
15 ('Jane', 'Smith', '1990-07-22', 'jane.smith@example.com'),
16 ('Mark', 'Williams', '1975-09-13', 'mark.williams@example.com'),
17 ('Anna', 'Brown', '1985-12-01', 'anna.brown@example.com'),
18 ('Tom', 'Davis', '1988-06-23', 'tom.davis@example.com'),
19 ('Lisa', 'Johnson', '1992-03-19', 'lisa.johnson@example.com'),
20 ('Steve', 'Martinez', '1983-08-07', 'steve.martinez@example.com'),
21 ('Linda', 'Garcia', '1978-11-30', 'linda.garcia@example.com'),
22 ('Paul', 'Roberts', '1969-04-25', 'paul.roberts@example.com'),
23 ('Nancy', 'Harris', '1982-09-15', 'nancy.harris@example.com'),
24 ('James', 'Clark', '1993-05-05', 'james.clark@example.com'),
25 ('Emily', 'Walker', '1987-07-18', 'emily.walker@example.com'),
26 ('Robert', 'Allen', '1985-10-20', 'robert.allen@example.com'),
27 ('Sophie', 'Young', '1976-11-29', 'sophie.young@example.com'),
28 ('Michael', 'Scott', '1991-12-24', 'michael.scott@example.com');
```

Sample Data for the Doctors Table

```

38 • INSERT INTO Doctors (FirstName, LastName, Specialty, ContactInfo)
39 VALUES
40 ('Emily', 'Stone', 'Cardiologist', 'emily.stone@example.com'),
41 ('Michael', 'Brown', 'Dermatologist', 'michael.brown@example.com'),
42 ('Sarah', 'Taylor', 'Neurologist', 'sarah.taylor@example.com'),
43 ('David', 'Anderson', 'Orthopedic', 'david.anderson@example.com'),
44 ('Laura', 'Lee', 'Pediatrician', 'laura.lee@example.com'),
45 ('Chris', 'Adams', 'Gastroenterologist', 'chris.adams@example.com'),
46 ('Daniel', 'Hall', 'Endocrinologist', 'daniel.hall@example.com'),
47 ('Jessica', 'Parker', 'Gynecologist', 'jessica.parker@example.com'),
48 ('Brian', 'Evans', 'Oncologist', 'brian.evans@example.com'),
49 ('Sophia', 'Nelson', 'Psychiatrist', 'sophia.nelson@example.com'),
50 ('Samuel', 'Wright', 'Urologist', 'samuel.wright@example.com'),
51 ('Oliver', 'Thomas', 'Pulmonologist', 'oliver.thomas@example.com'),
52 ('Amelia', 'Thompson', 'Ophthalmologist', 'amelia.thompson@example.com'),
53 ('Ethan', 'Mitchell', 'General Surgeon', 'ethan.mitchell@example.com'),
54 ('Olivia', 'Carter', 'Nephrologist', 'olivia.carter@example.com');
55

```

Sample Data for the Appointments Table

```

67 • INSERT INTO Appointments (PatientID, DoctorID, AppointmentDate)
68 VALUES
69 (1, 1, '2024-08-25'),
70 (2, 2, '2024-08-26'),
71 (3, 3, '2024-08-27'),
72 (4, 4, '2024-08-28'),
73 (5, 5, '2024-08-29'),
74 (6, 6, '2024-08-30'),
75 (7, 7, '2024-09-01'),
76 (8, 8, '2024-09-02'),
77 (9, 9, '2024-09-03'),
78 (10, 10, '2024-09-04'),
79 (11, 11, '2024-09-05'),
80 (12, 12, '2024-09-06'),
81 (13, 13, '2024-09-07'),
82 (14, 14, '2024-09-08'),
83 (15, 15, '2024-09-09');

```

Sample Data for the MedicalRecords Table


```

95 • INSERT INTO MedicalRecords (PatientID, RecordDate, Diagnosis, Treatment)
96 VALUES
97 (1, '2024-08-10', 'Hypertension', 'Medication A'),
98 (2, '2024-08-12', 'Diabetes', 'Insulin Therapy'),
99 (3, '2024-08-14', 'Migraine', 'Therapy A'),
100 (4, '2024-08-15', 'Asthma', 'Inhaler'),
101 (5, '2024-08-16', 'Arthritis', 'Physiotherapy'),
102 (6, '2024-08-18', 'Anxiety', 'Cognitive Behavioral Therapy'),
103 (7, '2024-08-19', 'High Cholesterol', 'Statin Therapy'),
104 (8, '2024-08-21', 'Hypothyroidism', 'Hormone Replacement Therapy'),
105 (9, '2024-08-23', 'Heart Disease', 'Angioplasty'),
106 (10, '2024-08-25', 'Obesity', 'Diet and Exercise'),
107 (11, '2024-08-26', 'Allergies', 'Antihistamine'),
108 (12, '2024-08-27', 'Back Pain', 'Chiropractic Therapy'),
109 (13, '2024-08-29', 'Depression', 'Antidepressants'),
110 (14, '2024-08-30', 'Insomnia', 'Sleep Therapy'),
111 (15, '2024-08-31', 'Eczema', 'Topical Steroids');

```

Sample Data for the Medications Table

```

121 • INSERT INTO Medications (MedicationName, Dosage)
122 VALUES
123 ('Medication A', '100mg'),
124 ('Insulin', '10ml'),
125 ('Therapy A', '30mg'),
126 ('Inhaler', '2 puffs'),
127 ('Physiotherapy', '1 hour'),
128 ('Statin', '20mg'),
129 ('Antihistamine', '10mg'),
130 ('Chiropractic Therapy', '30 minutes'),
131 ('Antidepressants', '50mg'),
132 ('Sleep Therapy', '1 hour'),
133 ('Hormone Replacement', '5mg'),
134 ('Angioplasty', 'Procedure'),
135 ('Diet Plan', 'N/A'),
136 ('Topical Steroids', '5mg'),
137 ('Cognitive Therapy', '1 hour');

```

Sample Data for the Prescriptions Table

```

150 • INSERT INTO Prescriptions (PatientID, MedicationID, PrescriptionDate)
151 VALUES
152 (1, 1, '2024-08-12'),
153 (2, 2, '2024-08-13'),
154 (3, 3, '2024-08-14'),
155 (4, 4, '2024-08-15'),
156 (5, 5, '2024-08-16'),
157 (6, 6, '2024-08-17'),
158 (7, 7, '2024-08-18'),
159 (8, 8, '2024-08-19'),
160 (9, 9, '2024-08-20'),
161 (10, 10, '2024-08-21'),
162 (11, 11, '2024-08-22'),
163 (12, 12, '2024-08-23'),
164 (13, 13, '2024-08-24'),
165 (14, 14, '2024-08-25'),
166 (15, 15, '2024-08-26');

```

4. Task 2 Queries: SQL Queries and Outputs

Query 1: List All Appointments for the Upcoming Week

This query enables administrators to have a schedule of all the appointments that have been set to take place in the subsequent one week (Vyawahare et al., 2023).

Verification Details:

1. **Logic:** The query uses `CURDATE()` to obtain the current date and `DATE_ADD(CURDATE(), INTERVAL 7 DAY)` to expand the range of the next seven days. It correctly selects the records from the Appointments table that are available from the current date to one week from now.
2. **Error Handling:** here is no need of error handling explicitly because the format of the AppointmentDate field has to be in the format of YYYY-MM-DD. The query also supposes that there are no null or incorrect values in the date field, it must be done in the database level.
3. **Scalability:** As the amount of appointments increases it also performs rather well because the query consists of a comparison operation. But, if the data increases, then indexing the AppointmentDate column would improve the query even more.
4. **Real-World Use:** In a real healthcare setting, this query will enable the management to schedule the human resource and other resources needed for the week in advance in order to meet the patients' appointments (Gamal, Barakat and Rezk, 2020).

1	/*Query 1: List all appointments for the upcoming week*/			
2	•	USE HealthcareDB;		
3				
4	•	SELECT * FROM Appointments		
5		WHERE AppointmentDate BETWEEN CURDATE() AND DATE_ADD(CURDATE(), INTERVAL 7 DAY);		
6				

Result Grid	Filter Rows:	Edit:	Export/Import:	Wrap Cell Content:
	AppointmentID	PatientID	DoctorID	AppointmentDate
▶	1	1	1	2024-08-25
	2	2	2	2024-08-26
	3	3	3	2024-08-27
	4	4	4	2024-08-28
	5	5	5	2024-08-29
	6	6	6	2024-08-30
*	NULL	NULL	NULL	NULL

Query 2: Find All Patients Prescribed a Specific Medication

This query identifies all patients who have been prescribed a particular medication (e.g., "Medication A").

Verification Details:

- Logic:** The query performs two JOIN operations:
 - o The first join is an inner join which is defined between the Patients and Prescriptions on the basis of PatientID.
 - o The second join ties Prescriptions and Medications tables based on MedicationID.
 - o It then narrows down the results to specific records such that the medication name in the record is exactly "Medication A".
- Efficiency:** It is possible to observe two joins in the query, which might be a problem in terms of performance if the tables become large. To increase scalability, indexing of PatientID and MedicationID would further be recommended to increase the performance. The database should also be create in such a way that there is referential integrity between these tables (by creating foreign keys).
- Real-World Use:** This query targets to enable the healthcare administrators to get details of the patients who are on a specific medication at a given time. It helps with monitoring of usage and checking on stock balance to ensure that it is adequate for use.
- Error Handling:** If the medication name is typed in wrong, or if in the data there are some typing mistakes, then the query will not produce any results. This shows why data entry validation should be done.

```
1  /*Query 2: Find all patients prescribed a specific medication*/
2  •  USE HealthcareDB;
3
4  •  SELECT Patients.FirstName, Patients.LastName
5     FROM Patients
6     JOIN Prescriptions ON Patients.PatientID = Prescriptions.PatientID
7     JOIN Medications ON Prescriptions.MedicationID = Medications.MedicationID
8     WHERE Medications.MedicationName = 'Medication A';
9
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	FirstName	LastName
▶	John	Doe

Query 3: List All Doctors and Their Specialties

This query retrieves a list of all doctors along with their specialties, assisting in resource allocation.

Verification Details:

1. **Logic:** The query is a SELECT statement that specifies the columns needed in the Doctors table and does not include any WHERE clause or joins. It is also based on the presumption that every doctor has a genuine specialty code recorded.
2. **Efficiency:** The query is fast because it is working on one table and does not involve any join or condition of any sort. However, for large data sets, indexing the Specialty column may prove useful if in future other filters will be applied to the doctors (for example, if applied filters will be the Specialty column in the future) (Samra, Li and Soh, 2020).
3. **Real-World Use:** This query assist to assign doctors to patients according to their specialisms. It is also useful in reporting the distribution of specialties in the healthcare facility.

1	/*Query 3: List all doctors along with their specialties*/
2	
3	• USE HealthcareDB;
4	
5	• SELECT FirstName, LastName, Specialty
6	FROM Doctors;
7	

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
	FirstName	LastName	Specialty	
▶	Emily	Stone	Cardiologist	
	Michael	Brown	Dermatologist	
	Sarah	Taylor	Neurologist	
	David	Anderson	Orthopedic	
	Laura	Lee	Pediatrician	
	Chris	Adams	Gastroenterologist	
	Daniel	Hall	Endocrinologist	
	Jessica	Parker	Gynecologist	
	Brian	Evans	Oncologist	
	Sophia	Nelson	Psychiatrist	
	Samuel	Wright	Urologist	
	Oliver	Thomas	Pulmonologist	
	Amelia	Thompson	Ophthalmologist	
	Ethan	Mitchell	General Surgeon	
	Olivia	Carter	Nephrologist	

5. Task 3 Queries: Business Analytics Queries and Outputs

Query 1: Average Number of Appointments Per Doctor

This query calculates the average number of appointments each doctor has handled, providing insight into doctor workloads.

Clarified Verification:

- **Use Case:** This query aid the healthcare administration to know if the doctors are overworked or they are idle. It makes it possible to modify the schedules in a way that seeks to even out the flow of patients and other booking appointments among the medical practitioners.
- **Performance Considerations:**
 - **Subquery Optimization** The inner query categorizes the appointments by the doctor and the number of appointments a particular doctor has. This subquery could be optimized by creating an indexed view, if the data is huge in size.

- o **Indexes:** Make sure that DoctorID in the Appointments table is indexed since this operation of grouping will be fast (delos reyes, 2016).
- **Edge Cases:**
 - o **Doctors with No Appointments:** To that extent, if there are any doctors who do not have any appointments, make certain that the query includes it. One of it is left join that will include all the doctors with zero appointments.
 - o **Multiple Locations:** If physicians practice in several facilities in large healthcare systems, location-based appointments may have to supplement the query to give a finer view.

```

1  /*Query 1: Calculate the average number of appointments per doctor*/
2
3  •  USE HealthcareDB;
4
5  •  SELECT DoctorID, AVG(AppointmentsPerDoctor) AS AvgAppointments
6  FROM (
7      SELECT DoctorID, COUNT(*) AS AppointmentsPerDoctor
8      FROM Appointments
9      GROUP BY DoctorID
10 ) AS DoctorAppointments
11 GROUP BY DoctorID
12 LIMIT 0, 1000;

```

DoctorID	AvgAppointments
1	1.0000
2	1.0000
3	1.0000
4	1.0000
5	1.0000
6	1.0000
7	1.0000
8	1.0000
9	1.0000
10	1.0000
11	1.0000

Query 2: Top 3 Most Commonly Prescribed Medications

This query identifies the top three most commonly prescribed medications.

Clarified Verification:

- **Use Case:** The query is also useful in understanding the commonly prescribed drugs and this is very important especially when it comes to ordering more stocks. It can also be used to analyze the prescriptive behavior and the trends of patients' treatments.
- **Performance Considerations:**
 - o **Optimization of Joins:** Just any other queries that require join, having index on MedicationID in both Medications and Prescriptions tables is mandatory.

- o **Efficient Limiting:** LIMIT 3 optimizes work because it only provides the first three medications, thus reducing the number of computations.
- **Edge Cases:**
 - o **Multiple Forms of Medications:** In some instances, the same medicines have different shapes (e. g. , a pill or a liquid). The query might need to be modified for prescription by active ingredients instead of having it only by the medication's name.

```

1  /*Query 2: Top 3 most commonly prescribed medications*/
2
3  • USE HealthcareDB;
4
5  • SELECT Medications.MedicationName, COUNT(*) AS PrescriptionCount
6     FROM Prescriptions
7     JOIN Medications ON Prescriptions.MedicationID = Medications.MedicationID
8     GROUP BY Medications.MedicationName
9     ORDER BY PrescriptionCount DESC
10    LIMIT 3;
11

```

MedicationName	PrescriptionCount
Medication A	1
Insulin	1
Therapy A	1

Query 3: Generate a Monthly Report of New Patients Added Over the Past Year

This query tracks the number of new patients added to the system each month over the past year.

Clarified Verification:

- **Use Case:** This query is used in segregating patient registrations by month, thus aiding in the monitoring of patient volumes in healthcare facilities. This data is useful in managing the resources that will be required in attending to the patients in future.
- **Performance Considerations:**
 - o **Date Indexing:** It is a good practice to index the DateOfBirth column, as the WHERE clause will be used on the DateOfBirth field and will be much faster due to the indexed field.
 - o **GROUP BY Optimization:** As the query includes the grouping of data by month it will be much faster if the index on the DateOfBirth column is updated and maintained properly.

- **Edge Cases:**
 - o **Seasonal Patterns:** In other cases, if the patient registrations are affected by the seasons strongly (for instance, flu seasons), the query results can be used in the modeling of the registered patients and adaptation of the healthcare resources.

```

1  /*Query 3: Generate a monthly report of new patients added to the system over the past year*/
2
3  •  USE HealthcareDB;
4
5  •  SELECT
6      DATE_FORMAT(DateOfBirth, '%Y-%m') AS MonthYear,
7      COUNT(*) AS NewPatients
8  FROM Patients
9  WHERE DateOfBirth >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)
10 GROUP BY MonthYear
11 ORDER BY MonthYear;
12
13

```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

MonthYear	NewPatients
-----------	-------------

6. Business Insights and Applications

Recording the registration of new patients on monthly basis is very crucial for efficient healthcare management to inform decision making in resource mobilization and management and patient flow and general operation. The following insights can be derived from the monthly report: The following insights can be derived from the monthly report:

Trend Analysis: Trends of patient registrations can be easily reflected by the administrators. For instance, high activity level of new patient registrations in some months could be explained by seasonal factors or changes in promotional campaigns, public health programmes, etc.

Resource Allocation: This way the management is able to understand when patient registrations are likely to be high, and therefore plan the staffing and resources accordingly. For instance, in the months when patient turnover is high more doctors, nurses or administrative personnel may be required.

Future Planning: The data can be used for forecasting future trends that would assist health care facilities to plan for expansion or contraction from the information in the database.

Operational Efficiency: Analyzing the data related to the registration of patients may help identify the problems in the process of onboarding patient. In case the trend shows fluctuations in the registration of patients across the particular months, then the healthcare facilities can look for the reasons and rectify the registration errors.

7. Challenges and Limitations

While this database system and query offer valuable insights, there are some challenges and limitations to consider: While this database system and query offer valuable insights, there are some challenges and limitations to consider:

Use of DateOfBirth Field: The DateOfBirth was employed as a stand-in for the registration day. Real life application would be better served if a RegistrationDate field is created to capture the date the patient was registered into the system to provide more reliability in reporting.

Data Accuracy: Some of the problems that can occur due to inaccuracies or omissions of data entry include wrong dates such as missing dates, wrong dates etc. To ensure the accuracy of the report data validation should be conducted and besides the data should be audited periodically.

Scalability: The query works fine for the small data but as the number of records in the Patients table increases the performance problems may occur. It is therefore recommended to index the DateOfBirth or RegistrationDate field to enhance performance with bigger databases.

External Factors: Fluctuations in the patients' registrations may be as a result of factors like public health campaigns, flu seasons, or even global health threats like a pandemic. The following factors should be borne in mind while analysing the data.

8. Conclusion

Therefore, this report stresses the importance of proper approach to designing databases and constructing queries in the sphere of healthcare. A good MySQL database allows administrators within the healthcare setting to store and access a lot of data, in turn promote proper decision making. This database developed in the course of this project for healthcare systems is centered on new patient registration, doctor schedules, and resource utilization.

Many to many relationships were maintained using primary and foreign keys to ensure data integrity and for efficient querying of patient, doctor, appointments, medications and prescriptions tables. Other queries such as tracking the upcoming appointments have been observed to be of great help for planning and management of resources within the system.

One more useful query is to find patients that are taking particular medications, allowing to control stock and patients' treatment at the same time. Also, the query to find the average number of appointments per doctor assists to schedule the work load and make it more effective since patients will not have to wait for long hours.

The number of new patients registered in a particular month to the health facility can be useful to the administrators in planning on the number of staffs or resources required at a certain time of the year. Though, at the present the database is efficient, in the future some modifications could be made: index frequently searched columns; add the new field `**RegistrationDate**`.

Therefore, the database aids efficient healthcare delivery by providing valuable information that enhances resource allocation, patients' management, and system performance.

Reference list

C., A.J., C., A.V. and E., N.S. (2018). Design and Implementation of a Hospital Database Management System (HDMS) for Medical Doctors. *International Journal of Computer Theory and Engineering*, [online] 10(1), pp.1–6. doi:<https://doi.org/10.7763/ijcte.2018.v10.1190>.

Celikel Cankaya, E. (2019). *Creative Commons — Attribution-NonCommercial-NoDerivatives 4.0 International — CC BY-NC-ND 4.0*. [online] Creativecommons.org. Available at: <http://creativecommons.org/licenses/by-nc-nd/4.0/> [Accessed 24 Aug. 2024].

delos reyes, edward (2016). *Hospital management system database design*. [online] Academia.edu. Available at: https://www.academia.edu/23018574/Hospital_management_system_database_design [Accessed 24 Aug. 2024].

Donaldson, M.S. and Lohr, K.N. (2019). *Health Databases and Health Database Organizations: Uses, Benefits, and Concerns*. [online] Nih.gov. Available at: <https://www.ncbi.nlm.nih.gov/books/NBK236556/> [Accessed 24 Aug. 2024].

Gamal, A., Barakat, S. and Rezk, A. (2020). Standardized electronic health record data modeling and persistence: A comparative review. *Journal of Biomedical Informatics*, [online] 114, p.103670. doi:<https://doi.org/10.1016/j.jbi.2020.103670>.

Muji, M., Ciupa, R.V., Dobru, D., Bica, C., Olah, P., Bacarea, V. and Marusteri, M. (2009). Database Design Patterns for Healthcare Information Systems. *IFMBE Proceedings*, [online] 26, pp.63–66. doi:https://doi.org/10.1007/978-3-642-04292-8_14.

Samra, H., Li, A. and Soh, B. (2020). G3DMS: Design and Implementation of a Data Management System for the Diagnosis of Genetic Disorders. *Healthcare*, [online] 8(3), p.196. doi:<https://doi.org/10.3390/healthcare8030196>.

Vyawhare, S.A., Bhushan, Dr.S., S. Kathane, Dr.M., Raut, R.R. and Tayde, S.R. (2023). DESIGN EFFICIENT CENTRALIZED PATIENT DATABASE SYSTEM FOR HEALTHY MONITORING OF DISEASE DIAGNOSIS IMPROVEMENT. *Sachin A. Vyawhare*, [online] 30. doi:<https://doi.org/10.53555/jptcp.v30i1.2391>.